Dataset Selection. Gehman et al. (2020) introduced the main dataset used to evaluate toxicity in **RealToxicityPrompts**. Shortly thereafter, Dhamala et al. (2021) introduced **BOLD**, which follows a similar structure as **RealToxicityPrompts** but uses more innocuous input prompts. Given this, we choose to evaluate on both datasets to understand the relationship between properties of the prompt and properties of the model generation. We note that other works, especially Abid et al. (2021), also demonstrate language model toxicity, but do not release a standardized dataset that has been used for measuring toxicity more broadly.

Problem Setting. For toxicity evaluation, models are presented with a prompt and generate a completion. In the case of **RealToxicityPrompts**, these prompts are drawn from OpenWebText (Gokaslan & Cohen, 2019), which is a collection of Internet text that replicates the training data of GPT-2 (Radford et al., 2019). To provide a stratified sample of different prompt toxicities, Gehman et al. (2020) ran the text in the corpus through PerspectiveAPI, binning into 4 bins ([0, .25), [0.25, 0.50), [0.50, 0.75), [0.75, 1.00]) and sampling 25k sentences from each. For **BOLD**, the prompts are instead drawn from Wikipedia, taking the first 6-9 words of an article that mention a (profession, gender, race, religion, or political ideology), which also implies that toxicity measurements for **BOLD** can be stratified by the associated social category. We note that in comparison to **RealToxicityPrompts**, the prompts in **BOLD** tend to be more neutral, so generating toxic text is even less justifiable in these contexts. We use PerspectiveAPI to measure toxicity in the model completions, reiterating the extensive caveats regarding the validity of PerpsectiveAPI that we discussed in §4.8: METRICS-TOXICITY.

6 Models

Model	Model Creator	Modality	# Parameters	Tokenizer	Window Size	Access	Total Tokens	Total Queries	Total Cost
J1-Jumbo v1 (178B)	AI21 Labs	Text	178B	AI21	2047	limited	327,443,515	591,384	$10,926
J1-Grande v1 (17B)	AI21 Labs	Text	17B	AI21	2047	limited	326,815,150	591,384	$2,973
J1-Large v1 (7.5B)	AI21 Labs	Text	7.5B	AI21	2047	limited	342,616,800	601,560	$1,128
Anthropic-LM v4-s3 (52B)	Anthropic	Text	52B	GPT-2	8192	closed	767,856,111	842,195	-
BLOOM (176B)	BigScience	Text	176B	BLOOM	2048	open	581,384,088	849,303	4,200 GPU hours
T0++ (11B)	BigScience	Text	11B	T0	1024	open	305,488,229	406,072	1,250 GPU hours
Cohere xlarge v20220609 (52.4B)	Cohere	Text	52.4B	Cohere	2047	limited	397,920,975	597,252	$1,743
Cohere large v20220720 (13.1B)[56]	Cohere	Text	13.1B	Cohere	2047	limited	398,293,651	597,252	$1,743
Cohere medium v20220720 (6.1B)	Cohere	Text	6.1B	Cohere	2047	limited	398,036,367	597,252	$1,743
Cohere small v20220720 (410M)[57]	Cohere	Text	410M	Cohere	2047	limited	399,114,309	597,252	$1,743
GPT-J (6B)	EleutherAI	Text	6B	GPT-J	2048	open	611,026,748	851,178	860 GPU hours
GPT-NeoX (20B)	EleutherAI	Text	20B	GPT-NeoX	2048	open	599,170,730	849,830	540 GPU hours
T5 (11B)	Google	Text	11B	T5	512	open	199,017,126	406,072	1,380 GPU hours
UL2 (20B)	Google	Text	20B	UL2	512	open	199,539,380	406,072	1,570 GPU hours
OPT (66B)	Meta	Text	66B	OPT	2048	open	612,752,867	851,178	2,000 GPU hours
OPT (175B)	Meta	Text	175B	OPT	2048	open	610,436,798	851,178	3,400 GPU hours
TNLG v2 (6.7B)	Microsoft/NVIDIA	Text	6.7B	GPT-2	2047	closed	417,583,950	590,756	-
TNLG v2 (530B)	Microsoft/NVIDIA	Text	530B	GPT-2	2047	closed	417,111,519	590,756	-
davinci (175B)	OpenAI	Text	175B	GPT-2	2048	limited	422,001,611	606,253	$8,440
curie (6.7B)	OpenAI	Text	6.7B	GPT-2	2048	limited	423,016,414	606,253	$846
babbage (1.3B)	OpenAI	Text	1.3B	GPT-2	2048	limited	422,123,900	606,253	$211
ada (350M)	OpenAI	Text	350M	GPT-2	2048	limited	422,635,705	604,253	$169
text-davinci-002	OpenAI	Text	Unknown	GPT-2	4000	limited	466,872,228	599,815	$9,337
text-curie-001	OpenAI	Text	Unknown	GPT-2	2048	limited	420,004,477	606,253	$840
text-babbage-001	OpenAI	Text	Unknown	GPT-2	2048	limited	419,036,038	604,253	$210
text-ada-001	OpenAI	Text	Unknown	GPT-2	2048	limited	418,915,281	604,253	$168
code-davinci-002	OpenAI	Code	Unknown	GPT-2	4000	limited	46,272,590	57,051	$925
code-cushman-001 (12B)	OpenAI	Code	12B	GPT-2	2048	limited	42,659,399	59,751	$85
GLM (130B)	Tsinghua University	Text	130B	ICE	2048	open	375,474,243	406,072	2,100 GPU hours
YaLM (100B)	Yandex	Text	100B	Yandex	2048	open	378,607,292	405,093	2,200 GPU hours

Table 5: **Models.** Description of the 30 models evaluated in this effort: provenance for this information is provided in Appendix I. The total tokens, queries, and cost refer to the overall costs we incur to evaluate the given model. For this reason, the costs for open models are measured in GPU hours, the costs for commercial models are measured in the monetary cost calculated using the current pricing scheme for each model, and the costs for the private models were covered by the model providers.

We evaluated 30 state-of-the-art language models varying in size, training procedure, organization, and access. These 30 can be grouped into 16 families (models within a family are identical up to model scale),

and further into 12 organizations that created the model. Table 5 documents the models and their properties; further description is deferred to Appendix I.

Model selection. As one of the pillars of this work, our aim was common understanding of language models. To achieve this ambition, we had to navigate the *access* conditions of different models (Liang et al., 2022). Some models were **open** (full model weights are available for download) (e.g. GPT-NeoX (20B), OPT (175B), BLOOM (176B)), some were **limited** API access (e.g. davinci (175B), J1-Jumbo v1 (178B), Cohere xlarge v20220609 (52.4B)), and some were **closed**, but the model owner provided us with access for this research effort (Anthropic-LM v4-s3 (52B), TNLG v2 (530B)). Unfortunately, we did not evaluate models that we could not access (e.g. Google's PaLM (Chowdhery et al., 2022) and LaMDA (Thoppilan et al., 2022), DeepMind's Gopher (Rae et al., 2021) and RETRO (Borgeaud et al., 2022)). We more extensively discuss missing models in §10.4: MISSING-MODELS. We encourage future work to evaluate these models to allow for common understanding, and to develop standards for research access to these models (Liang et al., 2022).

Model	Hardware
GPT-J (6B)	2×A100 (10.4%); 4×2080 Ti (89.6%)
GPT-NeoX (20B)	2×A100 (73.9%); 11×2080 Ti (26.1%)
T5 (11B)	2×A100 (59.1%); 8×2080 Ti (40.9%)
T0++ (11B)	2×A100 (1.1%); 8×2080 Ti (98.9%)
UL2 (20B)	2×A100 (3.5%); 16×2080 Ti (96.5%)
YaLM (100B)	8×A100
GLM (130B)	8×A100
OPT (66B)	8×A100
OPT (175B)	8×A100
BLOOM (176B)	8×A100

Table 6: **Hardware and compute for public models.** To perform inference on the public models, we used the Together Research Computer. At the time of this work, Together Research Computer connects clusters at several institutions. We mainly use NVIDIA GeForce RTX 2080 Ti GPUs and NVIDIA A100 GPUs to perform inference. If jobs were run on multiple hardware configurations, we report all configurations separated by ";" (with the percentage of GPU hours spent on each configuration).

Computational Resources. For models that required we provide computational resources, we used the Together Research Computer (Table 6).[58] Specifically, Together Research Computer employs pipeline parallelism to achieve high-throughput inference: the model is partitioned across GPUs and the GPUs communicate with each other the activations of the model to accomplish forward passes to get the predictions. Generally, Together Research Computer will dispatch inference jobs to idle GPUs. For models hosted via commercial APIs, the providers of these models graciously provided us sufficient API credits to evaluate their models as part of this effort.

Live systems. For both the private models and commercial APIs, we are evaluating live systems that may be regularly updated in some cases. Unfortunately, the practices of (publicly disclosed) *version control* and the corresponding *change logs* are uneven across model providers. For this reason, to the extent available, we report model versions (e.g. the version date for Cohere models).

The results we produce are specific to the model versions at the time we queried them: we record these timestamps in the results we release. Further, due the scale of our evaluation, it is possible that models change over the duration of our evaluation, but we assume they do not change.[59] To the extent the models/APIs change, we anticipate the change is fairly gradual, therefore the results are not subject to large shocks, but we encourage efforts to longitudinally monitor such changes to clarify their nature (cf. Chen et al., 2022). Similarly, we note that the models we evaluate may be *deprecated* at some point after our evaluation.

[58]Together Research Computer: `https://together.xyz`

[59]This may relate to observations of unexpected non-determinism in model behavior as reported in `https://twitter.com/OfirPress/status/1542610741668093952` and studied in Ruis et al. (2022).

Isolated technical issues. In general, we evaluate every model on every scenario (see Figure 4), and for every scenario we measure every metric (see Table 4). However, in a few isolated cases, we encounter issues/concerns that are generally attributable to specific model providers. We believe these issues are quite marginal in influencing the overall results (e.g. most models are not affected at all, and even affected models are affected on few scenarios).

First, for scenarios which require probabilities and not just completions (e.g. **HellaSwag**, **MS MARCO (regular)**, **The Pile**), we currently are not able to evaluate (possibly because the model itself fundamentally does not yield such probabilities) for T0++ (11B), T5 (11B), UL2 (20B), GLM (130B), and YaLM (100B). Second, for two models (J1-Jumbo v1 (178B), J1-Grande v1 (17B)), the API yields a tokenization error which currently prevents us from reporting results for **NaturalQuestions** (closed-book), though we expect this to be resolved shortly based on correspondence with the model providers. Third, the model probabilities for completions sampled with temperature zero are incorrect for models from AI21 Labs: for this reason, we do not report results for calibration, **MS MARCO (regular)**, and **MS MARCO (TREC)**. We explicitly acknowledge this in Figure 4, which fully explains why we evaluate 96% of the (model, scenario) pairs instead of 100%.

Model contamination. While we provide some information on the models we evaluate in Table 5, with further details in Appendix I, we emphasize that we have uneven and usually limited information on the training procedure for these models (see Liang et al., 2022). Consequently, we generally lack a precise characterization of, and access to, the training data for these models with notable exceptions including the ROOTS corpus for BLOOM (176B)[60] as well as the Pile corpus along with further training details for GPT-J (6B) and GPT-NeoX (20B) (see Biderman et al., 2022). Without such access and/or disclosure of contamination by model providers, we cannot verify whether the model was trained on the data we evaluate it on. This concern is heightened given we evaluate in a few-shot manner (as we discuss in §7: PROMPTING), so the model being trained on data drawn from this evaluation distribution, even beyond the exact evaluation instances, could compromise the legitimacy/purity of such few-shot evaluation. For this reason, we flag known instances of model contamination (i.e. where the model creators have disclosed possible contamination for a given scenario) when we report results, and compile this known evidence in Appendix G, though overall it is still largely limited. We encourage more proactive documentation/disclosure of model contamination, as well as the use of canaries or other methods for minimizing contamination, for future work on language model development to maintain the health of the evaluation ecosystem.

Fairness of evaluation. We prioritize standardization: models are evaluated on the same scenarios, for the same metrics, and with the same prompts for 5-shot prompting. However, the models themselves can require vastly different resources to produce, which are only partially accounted for by our measurements of efficiency as one class of costs. In this sense alone, the evaluation can be seen as unfair (cf. Linzen, 2020; Bommasani et al., 2021, §4.4). Even beyond this, particular models may be better suited for specific scenarios, metrics, prompts, and broader adaptation methods. As an example, we evaluate T0++ (11B) in a 5-shot fashion, whereas the model was designed for 0-shot prompting, and T5 (11B) similarly was neither designed for in-context learning nor to follow instructions in prompts. As one concrete recommendation as the language modeling space matures, we recommend model developers explicitly declare how their models should be evaluated and what the scope is of their generality/when models should be preferred. We believe this disclosure will help to strike a productive balance between the general-purpose possibilities for language models in the abstract and what is concretely sensible for a given model.

7 Adaptation via prompting

Adaptation transforms a raw language model into a system that makes predictions on test instances. In short, we use prompting as our adaptation method with 5 in-context examples (when in-context examples are included) as depicted in Figure 23. However, many lower-level details must be specified to fully define the adaptation procedure: we discuss important conceptual details here and defer the remainder to Appendix J. Table 7 provides example scenarios along with the full set of adaptation parameters.

[60] See `https://huggingface.co/spaces/bigscience-data/roots-search`.